SRM

INSTITUTE OF SCIENCE & TECHNOLOGY

(Deemed to be **University** u/s 3 of UGC Act, 1956)

DELHI NCR CAMPUS, GHAZIABAD, (U.P.)

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(FACULTY OF SCIENCE & HUMANITIES)

DEPARTMENT OF COMPUTER APPLICATIONS

PRACTICAL FILE

DATA ANALYSIS USING PYTHON PROGRAMMING

[PCA25C04J]

MCA (Gen AI) 1ST YEAR, 2 SEMESTER

Session: 2025-26

SUBMITTED TO :

Dr. Rajeev Kumar Sharma
Assistant Professor
Department of Computer Applications

SUBMITTED BY :

Student Reg No: RA2532242030041
Student Name: ADITYA SINGH
Department of Computer Applications

S R M INSTITUTE OF SCIENCE & TECHNOLOGY

DELHI NCR CAMPUS, MODINAGAR

(FACULTY OF SCIENCE & HUMANITIES)

DEPARTMENT OF COMPUTER APPLICATIONS

Register No. : **RA2532242030041**

BONAFIDE CERTIFICATE

Certified to be the Bonafide record of the work done by **ADITYA SINGH** of **MCA (Gen AI) 1ST Year/ 2 Semester** for the Award of **Master's Degree** in the **DEPARTMENT OF COMPUTER APPLICATIONS** in **Data Analysis Using Python Programming [PCA25C04J]** laboratory during the Academic Year **2025-26.**

SUBJECT IN – CHARGE **HEAD – COMPUTER APPLICATIONS**

Submitted for the University Examination held on _____.

INTERNAL EXAMINER - 1 **INTERNAL EXAMINER – 2**

INDEX

S.No.	Experiment Title	Date of Experiment	Signature
1	Write a program to implement the concept of Lists, Tuples, Dictionaries and sets in python programming.	11-12-2025	
2	Creating Class, Objects and Methods: Write a program in Python that takes the name, roll number, and marks of a student and calculates their percentage, and prints the grade to demonstrate class, object and methods in python.	11-12-2025	
3	Process Standard Streams: Write a function to demonstrate the read standard input and write to the standard output and standard error streams.	18-12-2025	
4	Command-line Arguments, Shell Variables: (a) Write a program to calculate the average of the n numbers of arguments using Command-line Arguments. (b) Write a program to print your name by using the shell variables	18-12-2025	
5	Python scripts here perform real tasks (a) Write a program to plot a barplot in python. (b) Write a program to extract all the links from the python official website.	08-01-2026	
6	Write a program to show the different types (Single, Multiple, Multilevel) Inheritance in python.	08-01-2026	
7	creating Thread Using Threading Module: Write a program to print the odd and even numbers by using thread.	29-01-2026	
8	Represent Compound Data using Python Create a function to find the fruits that start with a specific letter by using a compound data structure.	29-01-2026	
9	Lists, Tuples, Dictionaries: Create a function by using the list, tuples, and dictionaries.	05-02-2026	
10	Write a program to show the descriptive statistics using NumPy.	05-02-2026	
11	Write a program to illustrate Indexing Operators in data frames.	12-02-2026	
12	Write a program to illustrate Plots using Pandas and Matplotlib.	12-02-2026	
13	Write a program to build the GUI application with Tkinter	19-02-2026	
14	Write a program to Create Tables Using SQLite	19-02-2026	
15	Write a program to implement the concept of Database Connectivity.	26-02-2026	
16	Read and write data from/to files in Python Programs: (a) Read and write data from/to files in Python (b) Write a program to add data to the file and read the specific data from the file	26-02-2026	

Date:……./……./……

Experiment – 1

Experiment 1: Write a program to implement the concept of Lists, Tuples, Dictionaries and sets in python programming

Aim : To implement Lists, Tuples, Sets, and Dictionaries in Python.

Procedure :

1. Initialize a List and perform append/update operations.
2. Create a Tuple to store fixed data.
3. Define a Set to demonstrate the removal of duplicate values.
4. Use a Dictionary to store and access data using key-value pairs.

Source Code :

```
# List
my_list = ["Apple", "Banana", "Cherry"]
my_list.append("Mango")
my_list[1] = "Blueberry"
print("List:", my_list)
# Tuple
my_tuple = ("Red", "Green", "Blue")
print("Tuple:", my_tuple)
# Set
my_set = {10, 20, 30, 10, 20}
my_set.add(40)
print("Set:", my_set)
# Dictionary
my_dict = {"Brand": "Tesla", "Year": 2024}
my_dict["Year"] = 2026
print("Dictionary:", my_dict
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pc\Desktop\observation> python -u "c:\Users\pc\Desktop\observation\one.py"
List: ['Apple', 'Blueberry', 'Cherry', 'Mango']
Tuple: ('Red', 'Green', 'Blue')
Set: {40, 10, 20, 30}
Dictionary: {'Brand': 'Tesla', 'Year': 2026}
PS C:\Users\pc\Desktop\observation>
```

Date:…./……../……

Experiment – 2

Experiment 2: Creating Class, Objects and Methods: Write a program in Python that takes the name, roll number, and marks of a student and calculates their percentage, and prints the grade to demonstrate class, object and methods in python.

Aim :

To implement a Python program that calculates student percentages and grades using classes, objects, and methods.

Procedure :

1. Create a class named Student with an __init__ method to initialize data.
2. Define a method calculate_grade to compute the percentage and determine the grade.
3. Create an object of the class, pass user input, and call the method to display results.

Source Code :

```
class Student:
    def __init__(self, name, roll, marks):
        self.name = name
        self.roll = roll
        self.marks = marks
    def display_result(self):
        percentage = (sum(self.marks) / 300) * 100
        if percentage >= 90: grade = "A+"
        elif percentage >= 75: grade = "A"
        elif percentage >= 60: grade = "B"
        else: grade = "C"
        print(f"\nName: {self.name}")
        print(f"Roll No: {self.roll}")
        print(f"Percentage: {percentage:.2f}%")
        print(f"Grade: {
```

```
s_name = input("Enter Name: ")
s_roll = input("Enter Roll No: ")
s_marks = [int(input(f"Enter Marks {i+1}: ")) for i in range(3)]


std_obj = Student(s_name, s_roll, s_marks)
std_obj.display_result()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pc\Desktop\observation> python -u "c:\Users\pc\Desktop\observation\two.py"
Enter Name: Vivek Azad
Enter Roll No: 14
Enter Marks 1: 78
Enter Marks 2: 87
Enter Marks 3: 95

Name: Vivek Azad
Roll No: 14
Percentage: 86.67%
Grade: A
PS C:\Users\pc\Desktop\observation>
```

Date:……/……../……..

Experiment -3

Experiment 3: Process Standard Streams: Write a function to demonstrate the read standard input and write to the standard output and standard error streams.

Aim : To implement a Python function that reads from the standard input stream and writes to the standard output and standard error streams.

Procedure :

1. Import the sys module to access standard streams.
2. Use sys.stdin to read the user input.
3. Use sys.stdout to print regular messages or data.
4. Use sys.stderr to display error messages or warnings specifically.

Source Code :

```
import sys
def process_streams():
    sys.stdout.write("Enter your message: ")
    sys.stdout.flush()
    user_input = sys.stdin.readline()
    if user_input.strip() == "":
        sys.stderr.write("Error: Input cannot be empty!\n")
    else:
        sys.stdout.write(f"Standard Output: Received -> {user_input}")
process_streams()
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pc\Desktop\observation> python -u "c:\Users\pc\Desktop\observation\three.py"
Enter your message: hello bbu
Standard Output: Received -> hello bbu
PS C:\Users\pc\Desktop\observation>
```

Date:……/……../……..

Experiment - 4

Experiment 4: Command-line Arguments, Shell Variables : (a) Write a program to calculate the average of the n numbers of arguments using Command-line Arguments. (b) Write a program to print your name by using the shell variables.

Aim

(a) To calculate the average of n numbers passed via Command-line Arguments.

(b) To access and print a name stored in a Shell Variable.

Procedure

1. Use the sys.argv list to capture arguments passed during program execution.
2. Convert the string arguments to integers and calculate their average.
3. Use the os.environ dictionary to fetch and display the value of a shell variable.

Source Code

(a): Command-line Arguments :

```
import sys
args = sys.argv[1:]
if not args:
    print("No arguments provided.")
else:
    numbers = [int(x) for x in args]
    avg = sum(numbers) / len(numbers)
    print(f"Average: {avg}")
```

Source Code

(b): Shell Variables

```
import os
# Before running, set variable in terminal: export MY_NAME="Rahul"
name = os.environ.get("MY_NAME", "Variable Not Set")
print(f"Name from Shell: {name}")
```

Output

(a) Command-line Execution:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pc\Desktop\observation> python -u "c:\Users\pc\Desktop\observation\four.py"
No arguments provided.
PS C:\Users\pc\Desktop\observation>
```

(b) Shell Variable Execution:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pc\Desktop\observation> python -u "c:\Users\pc\Desktop\observation\four.py"
Name from Shell: Variable Not Set
PS C:\Users\pc\Desktop\observation>
```

Date:……/……../……..

Experiment - 5

Experimemt 5: Python scripts here perform real tasks (a)Write a program to plot a barplot in python. (b) Write a program to extract all the links from the python official website.

Aim

(a) To create a bar plot using the Matplotlib library.

(b) To extract all hyperlinks from the official Python website using BeautifulSoup.

Procedure

1. **Plotting:** Use matplotlib.pyplot to define categories and their corresponding values, then render them as a bar chart.
2. **Scraping:** Use the requests library to fetch the HTML content of a webpage and BeautifulSoup to parse and find all <a> tags with href attributes.

Source Code

(a): Bar Plot

Python

```
import matplotlib.pyplot as plt

languages = ['Python', 'Java', 'C++', 'JavaScript']

popularity = [100, 70, 60, 85]

plt.bar(languages, popularity, color='skyblue')

plt.xlabel("Programming Languages")

plt.ylabel("Popularity Score")

plt.title("Language Popularity")

plt.show()
```

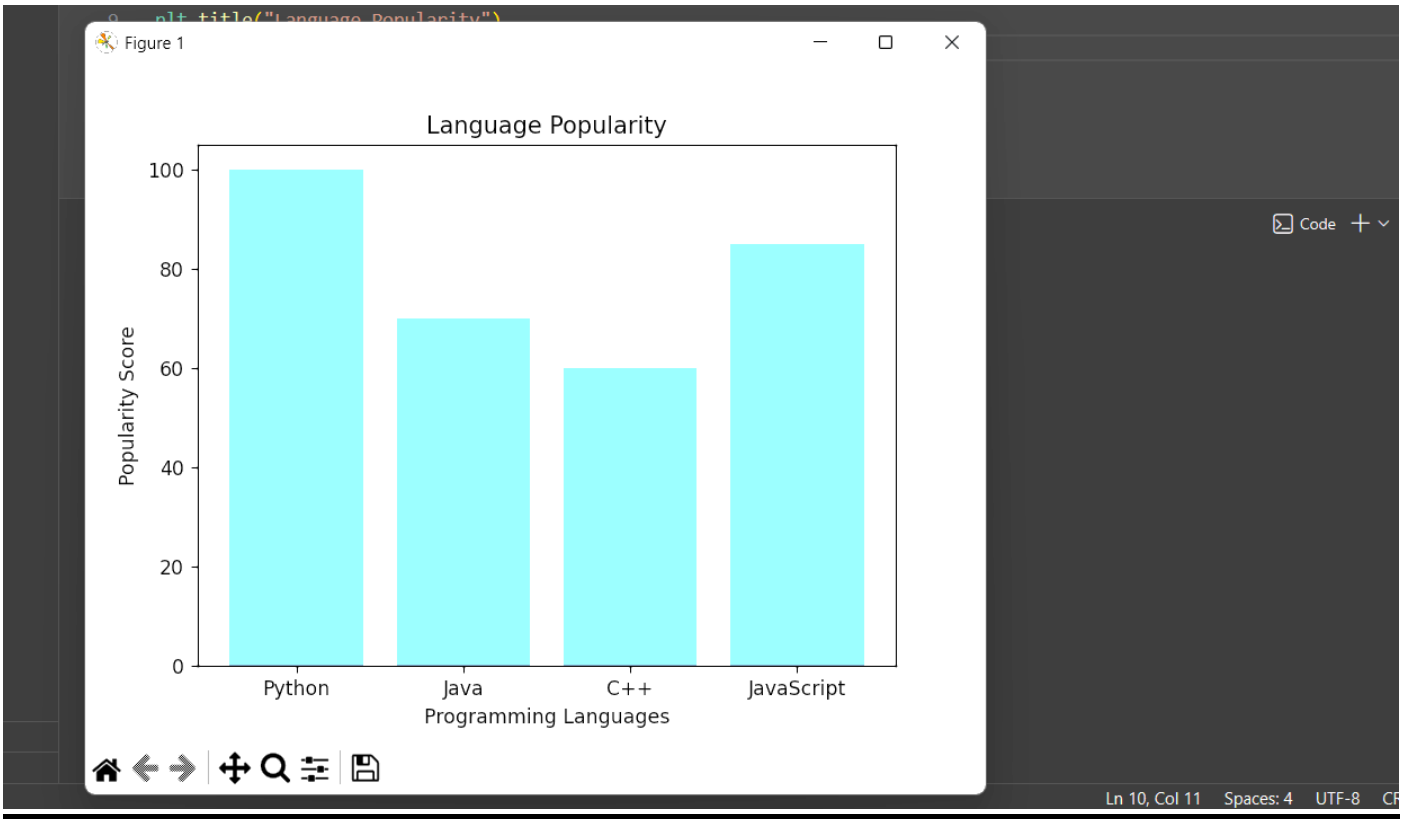
(b): Extract Links

Python

```
import requests

from bs4 import BeautifulSoup

url = "https://www.python.org"

response = requests.get(url)
```

```
soup = BeautifulSoup(response.text, 'html.parser')
for link in soup.find_all('a'):
    print(link.get('href'))
```

Output:



Date:……/……../……..

Experiment - 6

Experiment 6: Write a program to show the different types (Single,Multiple,Multilevel) Inheritance in python.

Aim

To implement and demonstrate Single, Multiple, and Multilevel Inheritance in Python.

Procedure

1. **Single Inheritance:** Create a child class that inherits from one parent class.
2. **Multiple Inheritance:** Create a child class that inherits from two or more independent parent classes.
3. **Multilevel Inheritance:** Create a derived class that inherits from another derived class (forming a chain).

Source Code

```
# 1. Single Inheritance
class Parent:
    def func1(self):
        print("This is the parent class.")
class Child(Parent):
    def func2(self):
        print("This is the child class.")
# 2. Multiple Inheritance
class Father:
    def father_power(self):
        print("Strength from Father.")
class Mother:
    def mother_power(self):
```

```
print("Wisdom from Mother.")
class Son(Father, Mother):
    def son_power(self):
        print("Son has both.")
# 3. Multilevel Inheritance
class Grandfather:
    def age1(self):
        print("Grandfather's property.")
class Father_ML(Grandfather):
    def age2(self):
        print("Father's property.")
class Son_ML(Father_ML):
    def age3(self):
        print("Son's property.")
# Execution
print("--- Single Inheritance ---")
obj1 = Child()
obj1.func1()
obj1.func2()
print("\n--- Multiple Inheritance ---")
obj2 = Son()
obj2.father_power()
obj2.mother_power()
print("\n--- Multilevel Inheritance ---")
obj3 = Son_ML()
obj3.age1()
obj3.age2()
obj3.age3()
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   Code

PS C:\Users\pc\Desktop\observation> python -u "c:\Users\pc\Desktop\observation\six.py"
--- Single Inheritance ---
This is the parent class.
This is the child class.

--- Multiple Inheritance ---
Strength from Father.
Wisdom from Mother.

--- Multilevel Inheritance ---
Grandfather's property.
Father's property.
Son's property.
PS C:\Users\pc\Desktop\observation>
```

Date:……/……../……..

Experiment - 7

Experiment 7: Creating Thread Using Threading Module: Write a program to print the odd and even numbers by using thread.

Aim : To design and implement a Python program that utilizes the threading module to concurrently print even and odd numbers within a specified range

Procedure :

- **Define the functions:** Create two distinct functions—one to iterate through even numbers and another **Import the module:** Bring in the threading module to access thread creation tools. for odd numbers.
- **Initialize Threads:** Create two thread objects, assigning each function as a target.
- **Start Execution:** Use the .start() method to begin the thread execution.
- **Synchronization:** Use the .join() method to ensure the main program waits for both threads to finish before exiting.

Code :

```
import threading
import time
def print_evens(limit):
    for i in range(0, limit + 1, 2):
        time.sleep(0.1) # Simulating work/delay
        print(f"Even Thread: {i}")
def print_odds(limit):
    for i in range(1, limit + 1, 2):
        time.sleep(0.1) # Simulating work/delay
        print(f"Odd Thread: {i}")

if __name__ == "__main__":
    n = 10  # Range limit
```

```
t1 = threading.Thread(target=print_evens, args=(n,))
t2 = threading.Thread(target=print_odds, args=(n,))

t1.start()
t2.start()
t1.join()
t2.join()
print("Finished printing all numbers.")
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pc\Desktop\observation> python -u "c:\Users\pc\Desktop\observation\seven.py"
Odd Thread: 1
Even Thread: 0
Odd Thread: 3
Even Thread: 2
Odd Thread: 5
Even Thread: 4
Odd Thread: 7
Even Thread: 6
Odd Thread: 9
Even Thread: 8
Even Thread: 10
Finished printing all numbers.
PS C:\Users\pc\Desktop\observation>
```

Date:……/……../…….

Experiment - 8

Experiment 8: Represent Compound Data using Python Create a function to find the fruits that start with a specific letter by using a compound data structure.

Aim :

To create a Python function that filters and identifies fruits from a compound data structure based on a specific starting letter.

Procedure :

- **Define the Data:** Create a list named fruit_inventory containing dictionaries. Each dictionary stores metadata for a fruit.
- **Create the Function:** Define find_fruits_by_letter which takes the list and a target letter as arguments.
- **Iterate and Filter:** Use a loop to check the 'name' key of each dictionary.
- **Case Normalization:** Convert both the fruit name and the input letter to lowercase using .lower() to ensure the search is case-insensitive.
- **Return Results:** Store matching fruit names in a list and return them.

Code :-

```
fruit_inventory = [
    {"name": "Apple", "color": "Red", "taste": "Sweet"},
    {"name": "Banana", "color": "Yellow", "taste": "Sweet"},
    {"name": "Apricot", "color": "Orange", "taste": "Tart"},
    {"name": "Blueberry", "color": "Blue", "taste": "Sweet"},
    {"name": "Cherry", "color": "Red", "taste": "Tart"},
    {"name": "Avocado", "color": "Green", "taste": "Nutty"}
]


def find_fruits_by_letter(data, letter):
    """
    Filters fruits that start with a specific lett
```

```
    """
    # Use list comprehension for a clean, Python search
    results = [fruit["name"] for fruit in data if
    fruit["name"].lower().startswith(letter.lower())]


    return results
search_letter = "A"
found_fruits = find_fruits_by_letter(fruit_inventory, search_letter)


print(f"Fruits starting with '{search_letter}':")
if found_fruits:
    for fruit in found_fruits:
        print(f"- {fruit}")
else:
    print("No fruits found.")
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pc\Desktop\observation> python -u "c:\Users\pc\Desktop\observation\eight.py"
Fruits starting with 'A':
- Apple
- Apricot
- Avocado
PS C:\Users\pc\Desktop\observation>
```

Experiment - 9

Experiment 9 : Lists, Tuples, Dictionaries: Create a function by using the list, tuples, and dictionaries.

Aim :

To create a Python function that integrates lists, tuples, and dictionaries to manage and retrieve student information.

Procedure :

1. **Initialize a List:** Create a list of student names to iterate through.
2. **Define Tuples:** Use tuples to store "frozen" information (ID, Major) that shouldn't change accidentally.
3. **Construct a Dictionary:** Use the student names as **keys** and the tuples as **values**.
4. **Create a Search Function:** Define a function that takes a name, looks it up in the dictionary, and returns a formatted string.

Code :

```
def manage_student_records():
    student_names = ["Alice", "Bob", "Charlie"]
    alice_info = (101, "Physics")
    bob_info = (102, "Mathematics")
    charlie_info = (103, "Biology")
    student_registry = {
        student_names[0]: alice_info,
        student_names[1]: bob_info,
        student_names[2]: charlie_info
    }
    return student_registry
def get_student_details(registry, name):
    """Function to retrieve data from the compound structure."""
    if name in registry:
```

```
    student_id, major = registry[name] # Unpacking the tuple
        return f"Student: {name} | ID: {student_id} | Major: {major}"
    else:
        return "Student not found."

# Execution
registry_data = manage_student_records()

# Example: Finding Bob
print(get_student_details(registry_data, "Bob"))
# Example: Finding Alice
print(get_student_details(registry_data, "Alice"))
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pc\Desktop\observation> python -u "c:\Users\pc\Desktop\observation\nine.py"
Student: Bob | ID: 102 | Major: Mathematics
Student: Alice | ID: 101 | Major: Physics
PS C:\Users\pc\Desktop\observation>
```

Experiment - 10

Experiment 10 : Write a program to show the descriptive statistics using NumPy.

Aim :

To implement a Python program using the NumPy library to calculate fundamental descriptive statistics (Mean, Median, Variance, and Standard Deviation) for a given dataset.

Procedure :

1. **Import NumPy:** Use import numpy as np to access the library.
2. **Create Dataset:** Initialize a NumPy array with a set of numerical values.
3. **Calculate Central Tendency:** Use np.mean() and np.median() to find the average and the middle value.
4. **Calculate Dispersion:** Use np.var() for variance and np.std() for standard deviation to measure how spread out the data is.
5. **Calculate Range:** Find the difference between np.max() and np.min().
6. **Display Results:** Print the calculated metrics clearly.

Code :

```
import numpy as np

def calculate_descriptive_stats(data_list):

    data = np.array(data_list)

    stats = {

        "Mean": np.mean(data),

        "Median": np.median(data),

        "Minimum": np.min(data),

        "Maximum": np.max(data),

        "Range": np.ptp(data),  # ptp stands for 'peak to peak'

        "Variance": np.var(data),

        "Std Deviation": np.std(data)}

    return stats
```

```
scores = [85, 90, 78, 92, 88, 76, 95, 89, 84, 82]
results = calculate_descriptive_stats(scores)
print("--- Descriptive Statistics ---")
for key, value in results.items():
    print(f"{key:15}: {value:.2f}")
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pc\Desktop\observation> python -u "c:\Users\pc\Desktop\observation\ten.py"
--- Descriptive Statistics ---
Mean           : 85.90
Median         : 86.50
Minimum        : 76.00
Maximum        : 95.00
Range          : 19.00
Variance       : 33.09
Std Deviation  : 5.75
PS C:\Users\pc\Desktop\observation>
```

Date:……/……../……..

Experiment – 11

Experiment 11:Write a program to illustrate Indexing Operators in data frames

AIM : To demonstrate different methods of **selecting and accessing data in a DataFrame using the Pandas library in Python**, including column selection, label-based indexing using `.loc`, and position-based indexing using `.iloc`.

Procedure :

- Import the Pandas library.
- Create a dictionary containing product details (Product, Price, Stock).
- Convert the dictionary into a DataFrame with custom index labels.
- Display the original DataFrame.
- Select the **Price** column using square brackets [].
- Use **.loc** to access rows and columns using labels.
- Use **.iloc** to access rows and columns using index positions.

Code :

```
import pandas as pd

    # Creating a sample DataFrame

    data = {

        'Product': ['Laptop', 'Mouse', 'Monitor', 'Keyboard', 'Webcam'],

        'Price': [1200, 25, 300, 75, 50],

        'Stock': [15, 100, 45, 60, 20]

    }

    df = pd.DataFrame(data, index=['ID1', 'ID2', 'ID3', 'ID4', 'ID5'])


    print("Original DataFrame:")

    print(df)

    print("-" * 30)

    # 1. Using Standard Brackets [] - Best for Columns

    print("Select single column (Price):")
```

```
print(df['Price'].head(2))
# 2. Using .loc (Label-based) - Access by name
# Syntax: df.loc[row_label, col_label]
print("\nAccess row 'ID3' using .loc:")
print(df.loc['ID3'])
print("\nAccess 'ID1' to 'ID2' for 'Product' and 'Stock':")
print(df.loc['ID1':'ID2', ['Product', 'Stock']])
# 3. Using .iloc (Integer-based) - Access by position
# Syntax: df.iloc[row_index, col_index]
print("\nAccess first row (index 0) using .iloc:")
print(df.iloc[0])
print("\nAccess first two rows and first two columns using .iloc slicing:")
print(df.iloc[0:2, 0:2])
```

Output :

Original DataFrame:

	Product	Price	Stock
ID1	Laptop	1200	15
ID2	Mouse	25	100
ID3	Monitor	300	45
ID4	Keyboard	75	60
ID5	Webcam	50	20

Select single column (Price):

```
ID1    1200
ID2      25     dytpe: int64
```

Access row 'ID3' using .loc:

```
Product      Monitor
Price        300
Stock        45
Name: ID3, dytpe: object
```

Access 'ID1' to 'ID2' for 'Product' and 'Stock':

	Product	Stock
ID1	Laptop	15
ID2	Mouse	100

Access first row (index 0) using .iloc:

```
Product      Laptop
Price        1200
Stock        15
Name: ID1, dytpe: object
```

Access first two rows and first two columns using .iloc slicing:

	Product	Price
ID1	Laptop	1200
ID2	Mouse	25

Date:……/……../……..

Experiment – 12

Experiment 12: Write a program to illustrate Plots using Pandas and Matplotlib.

AIM: To illustrate different types of plots using Pandas and Matplotlib in Python.

Procedure :

- Import the Pandas and Matplotlib libraries.
- Create sample data for **Year, Sales, and Profit**.
- Convert the data into a **Pandas DataFrame**.
- Use df.plot() to create **line, bar, and scatter plots**.
- Display the graphs using plt.show().

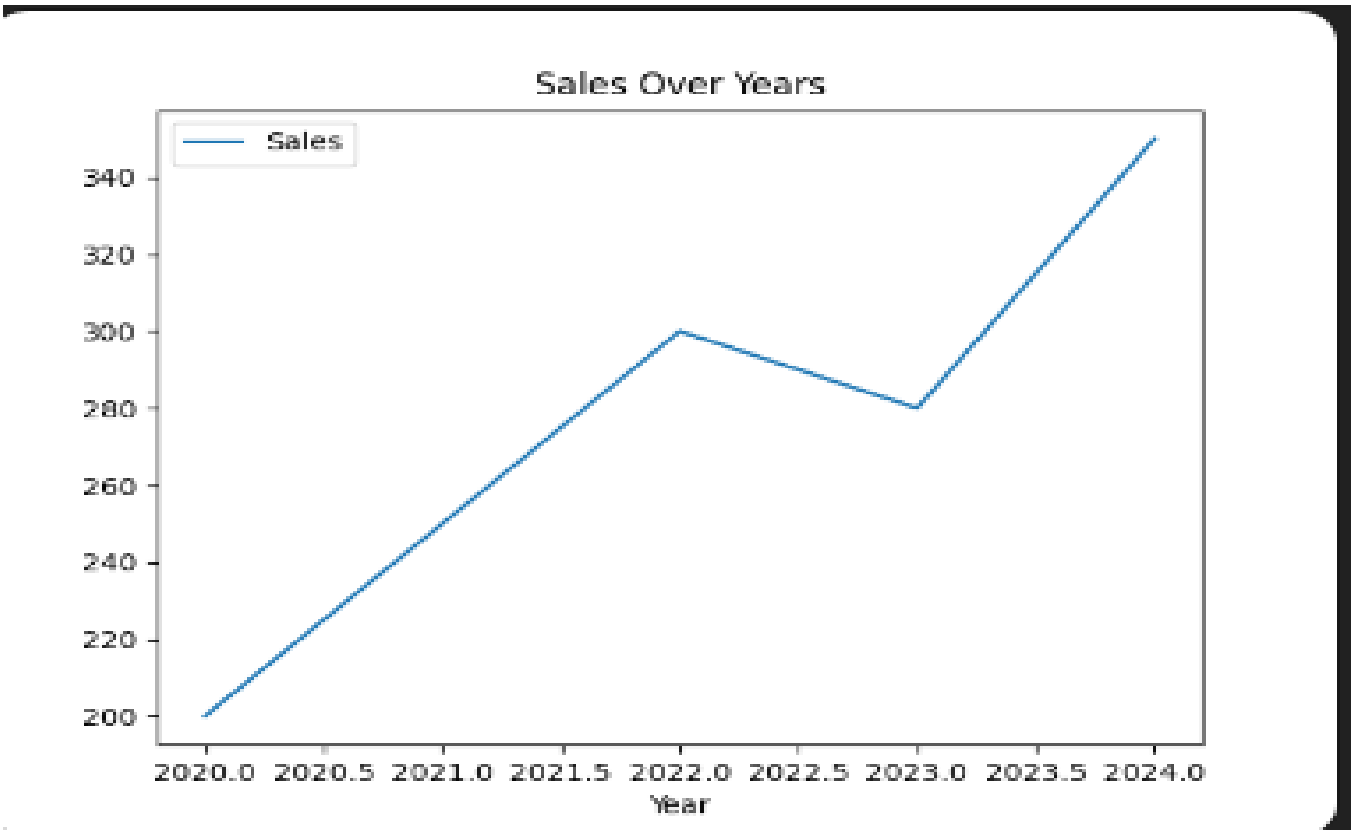
Code :

```
import pandas as pd

import matplotlib.pyplot as plt


# Creating sample data

data = {

    'Year': [2020, 2021, 2022, 2023, 2024],

    'Sales': [200, 250, 300, 280, 350],

    'Profit': [50, 60, 80, 70, 90]

}


# Creating DataFrame

df = pd.DataFrame(data)


# Line Plot

df.plot(x='Year', y='Sales', kind='line', title='Sales Over Years')

plt.show()
```

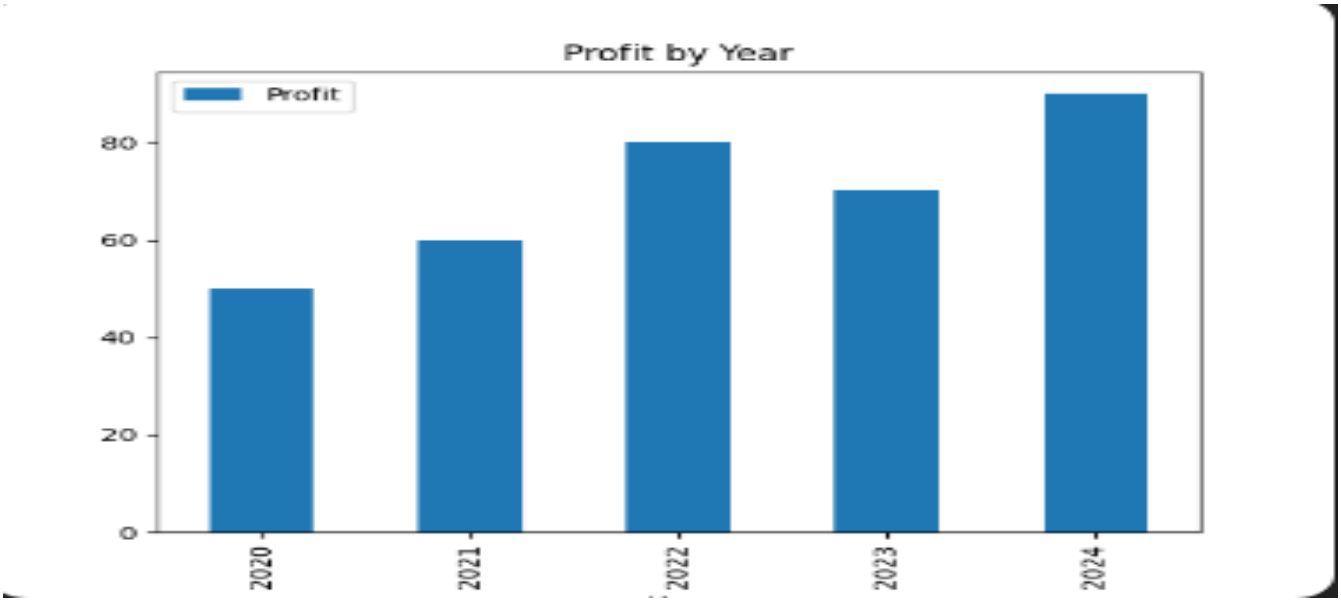
```
# Bar Plot
df.plot(x='Year', y='Profit', kind='bar', title='Profit by Year')
plt.show()

# Scatter Plot
df.plot(x='Sales', y='Profit', kind='scatter', title='Sales vs Profit')
plt.show()
```

Output :

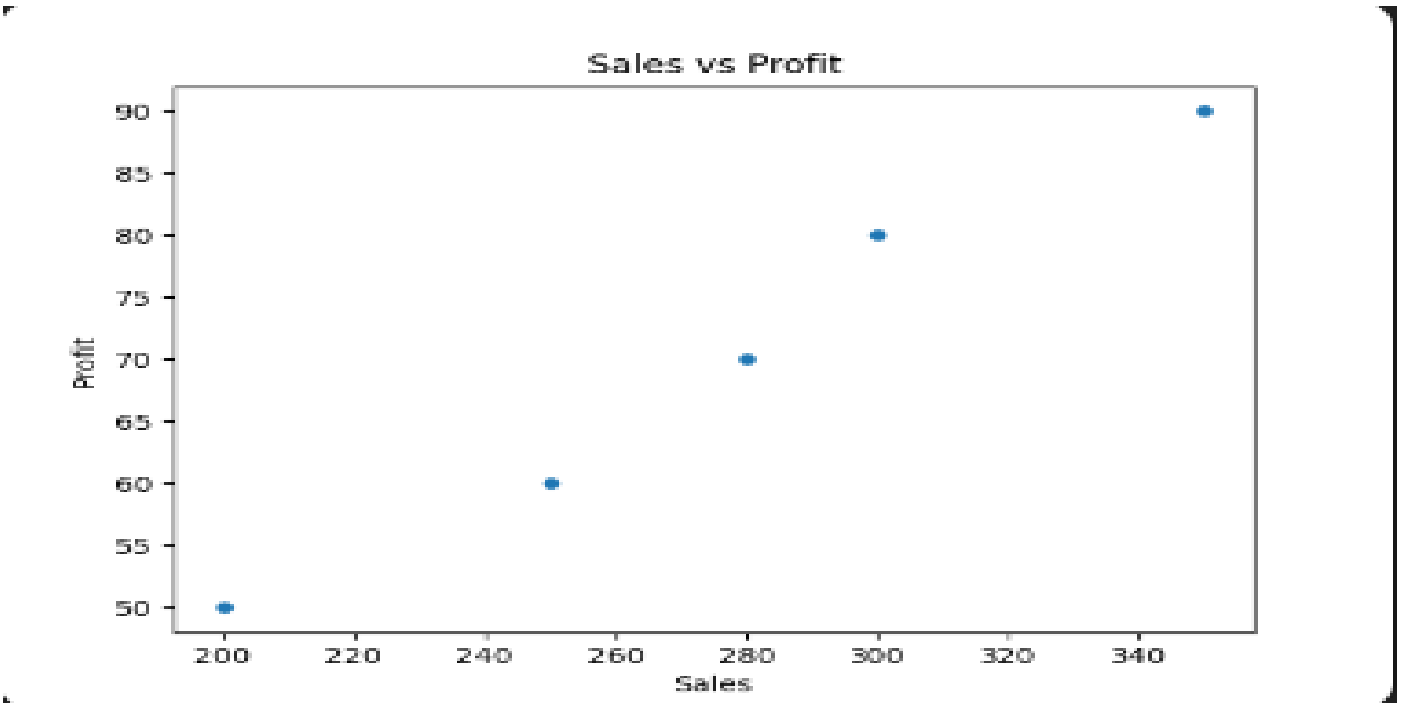
1. Line Plot



2. Bar Plot



3 Scatter plot



Experiment – 13

Experiment 13: Write a program to build the GUI application with Tkinter

AIM: To build a simple GUI application using Tkinter in Python.

.**Procedure :**

- Import the Tkinter library.
- Create the main window using Tk().
- Set the window title and size.
- Create a **label** and a **button** widget.
- Define a function that changes the label text when the button is clicked.
- Run the GUI application using mainloop().

Code :

```
import tkinter as tk

# Create main window

root = tk.Tk()

root.title("Simple GUI Application")

root.geometry("300x200")


# Function for button click

def show_message():

    label.config(text="Hello! Welcome to Tkinter GUI")


# Create label

label = tk.Label(root, text="Click the button", font=("Arial", 12))

label.pack(pady=20)


# Create button

button = tk.Button(root, text="Click Me", command=show_message)
```

```
button.pack()

# Run the application
root.mainloop()
```

Output :

Date:……/……../……..

Experiment – 14

Experiment 14: Write a program to Create Tables Using SQLite

AIM: To create a table in a database using SQLite with Python.

.Procedure :

- Import the `sqlite3` module in Python.
- Connect to the SQLite database using `connect()`.
- Create a cursor object to execute SQL commands.
- Use the `CREATE TABLE` SQL command to create a table.
- Commit the changes and close the database connection.

Code :

```
import sqlite3

# Connect to SQLite database (creates database if it does not exist)

conn = sqlite3.connect("student.db")


# Create a cursor object

cursor = conn.cursor()


# Create table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Students (

    ID INTEGER PRIMARY KEY,

    Name TEXT,

    Age INTEGER,

    Course TEXT

)

""")


print("Table created successfully")
```

```
# Save changes and close connection
conn.commit()
conn.close()
```

Output :

```
Table created successfully
```

Experiment – 15

Experiment 15: Write a program to implement the concept of Database Connectivity.

AIM: To implement the concept of **database connectivity using SQLite in Python.**

.Procedure :

- Import the sqlite3 module in Python.
- Connect to the database using connect().
- Create a cursor object to execute SQL commands.
- Create a table and insert records.
- Retrieve and display the records from the database.
- Commit changes and close the connection.

Code :

```
import sqlite3


# Connect to the database

conn = sqlite3.connect("student.db")


# Create cursor object

cursor = conn.cursor()


# Create table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Students (

    ID INTEGER PRIMARY KEY,

    Name TEXT,

    Age INTEGER

)

""")
```

```
# Insert data
cursor.execute("INSERT INTO Students (Name, Age) VALUES ('Aditya', 21)")
cursor.execute("INSERT INTO Students (Name, Age) VALUES ('Rahul', 22)")

# Fetch data
cursor.execute("SELECT * FROM Students")
rows = cursor.fetchall()

# Display records
for row in rows:
    print(row)

# Commit and close connection
conn.commit()
conn.close()
```

Output :

```
(1, 'Aditya', 21)
(2, 'Rahul', 22)
```

Experiment – 16

Experiment 16: Read and write data from/to files in Python Programs :

a. Read and write data from/to files in Python

b. Write a program to add data to the file and read the specific data from the file.

AIM: (a) To read and write data from/to a file using **file handling in Python**.

(b) To append data to a file and read specific data from the file using **file handling in Python**.

.Procedure :

- Open a file in **write (w) mode** using file handling in Python and write data into the file using write().
- Close the file after writing the data.
- Open the same file in **append (a) mode** to add new data without deleting the existing content.
- Use the write() function to add additional data to the file.
- Open the file in **read (r) mode** to access the stored data.
- Read the contents using read() or readlines() functions.
- Display the required or specific data from the file and then close the file.

Code :

Source Code : (a)

```
# Writing data to a file

file = open("sample.txt", "w")

file.write("Hello, this is a Python file handling example.\n")

file.write("Learning to read and write files.")

file.close()


# Reading data from the file

file = open("sample.txt", "r")

content = file.read()
```

```
print(content)
file.close()
```

Source Code : (b)

```
# Adding data to the file
file = open("data.txt", "a")
file.write("Aditya\n")
file.write("Rahul\n")
file.write("Priya\n")
file.close()
# Reading specific data from the file
file = open("data.txt", "r")
lines = file.readlines()
print("Second line from the file:")
print(lines[1])
file.close()
```

Output :

(a)

```
Hello, this is a Python file handling example.
Learning to read and write files.
```

(b)

```
Second line from the file:
Rahul
```